



Detection of crawler traps: formalization and implementation—defeating protection on internet and on the TOR network

Baptiste David¹ · Maxence Delong¹ · Eric Filiol^{2,3}

Received: 7 September 2020 / Accepted: 16 March 2021 / Published online: 1 April 2021
© The Author(s), under exclusive licence to Springer-Verlag France SAS, part of Springer Nature 2021

Abstract

In the domain of web security, websites strive to prevent themselves from data gathering performed by automatic programs called bots. In that way, crawler traps are an efficient brake against this kind of programs. By creating similar pages or random content dynamically, crawler traps give fake information to the bot and resulting by wasting time and resources. Nowadays, there is no available bots able to detect the presence of a crawler trap. Our aim was to find a generic solution to escape any type of crawler trap. Since the random generation is potentially endless, the only way to perform crawler trap detection is on the fly. Using machine learning, it is possible to compute the comparison between datasets of webpages extracted from regular websites from those generated by crawler traps. Since machine learning requires to use distances, we designed our system using information theory. We considered widely used distances compared to a new one designed to take into account heterogeneous data. Indeed, two pages does not have necessary the same words and it is operationally impossible to know all possible words by advance. To solve our problematic, our new distance compares two webpages and the results showed that our distance is more accurate than other tested distances. By extension, we can say that our distance has a much larger potential range than just crawler traps detection. This opens many new possibilities in the scope of data classification and data mining.

Keywords Crawler trap · Information distance · Bots · TOR network

1 Introduction

For most people, surfing the web is done via a web browser where it is just about to click on a few links. But the ecosystem is not only made of humans. Indeed, there are automatic programs, called bots, that allow you to automatically browse websites. Browsing motivations are multiple. Indeed, they can be used for indexing and archiving, search engine

optimization, content retrieval, user information collection — with or without the user's agreement — search for non-posted and potentially confidential documents, intelligence gathering for attack preparation... Usually, bots are efficient and largely faster than humans to collect all information. If the motivations of such tools are more or less legitimate, it is understandable that some sites do not wish to be analyzed in such a way.

To counter these unwanted automatic analysts, websites have implemented different strategies. From the detection of bots with basic methods, to challenges offered that only humans (Turing test) can solve (or supposed to be), various methods have emerged across the last years. In a few cases, the most advanced ones have the ability to neutralize the importune bot. The main technique used to deceive and hinder bots' operations is generically described as “Crawler traps” [1]. As always in IT security, what refers to detection measures and protection on one side, also refers to bypass techniques symmetrically. Hence the aim of our paper is to formalize the concept of crawler trap detection and manage-

✉ Eric Filiol
eric.filiol@univ-ubs.fr

Baptiste David
delong@et.esiea.fr

Maxence Delong
david@et.esiea.fr

¹ Laboratoire de Virologie et de cryptologie opérationnelles, ESIEA, Laval, France

² Department of Cyberdefense, ENSIBS, Vannes, France

³ Higher School of Economics, Moscow, Russian Federation

ment, to present different techniques capable of interfacing with anti-bot countermeasures and to present our detection results.

More than trying to be stealthy (it would result in reducing the bot's performance and therefore losing its main interest), we want to counter the measures put in place to neutralize bots. The objective is to have ways to pass where the other widely used bots generally stop and fail, trapped by the website's defenses. Our whole problem is to have the operational means to be able to detect whenever a website traps our bot and therefore to be able to avoid it. Surprisingly, there are neither no research no real public operational solutions on the subject, which means we are going to explore a new field.

This paper is therefore organized to address this problem. Section 2 presents the state of the art on the different website protection mechanisms. Section 3 addresses the formalization we have considered and presents the solutions implemented based on the theoretical aspects we have considered. Section 4 presents the results we have obtained from our solutions during a large number of experiments both on the Internet and on the TOR network (aka Darkweb). Section 5 presents in which context it is possible to perform detection with our solutions. Finally, Sect. 6 resumes our approach and possible uses.

2 State of art

As far as web bots are concerned, there is a great diversity of uses. If the majority of bots as chatbots [2] or Search Engine Optimization (SEO) bots as Googlebot [3] are very helpful for websites owners, some of them are malicious. The purpose of malicious bots can be multiple. Among many example, we can mention spambots which collect email addresses to prepare phishing campaigns. Subsequently to fight against botnets' activities, many security measures have been developed.

Most solutions degrade the user experience or come to be impractical from a user point of view. An example of a repressive solution is the IP banishment. If the web server guessed that a bot is crawling it, an error code can be returned on legitimate webpages thus disallowing content access. In the case where the website has incorrectly guessed and real user has been blocked, the frustration is high. Another example of common impractical security on the web are captchas [4]. Indeed, if originally, most of captchas were used to validate specific operation where a human user was mandatory (online payment, registration...), nowadays, websites more and more condition the access to the total content of the website by resolving a captcha first. But this is not a sufficient solution since there exists practical ways to automatically bypass them [5]. As a consequence, other security systems have been developed to be more efficient.

Usually, the *official* way to prevent web-crawlers is to create a file named "*robot.txt*" put at the root of the website. This file is used to reference all the sections of a website that legitimate bots should not crawl for indexing. As a result, traps are referenced in it to prevent legitimate bots to fall in it. This file is only declarative and there is no deeper protection to forbid crawling. In a way, it is a gentleman's agreement that determined bots shamelessly bypass. Moreover, the "*robot.txt*" gives direct information to the attacker on sensitive parts of the website.

The new trend to provide real protection is focused on systems called "*spider trap*" [6] or "*crawler trap*" [1]. The goal is to ambush the malicious bot on a dedicated trap inside the website. The intent is to imprison the bot in a specific place where it is supposed continuing to operate endless false information (or the same one, again and again) in order to finally waste time and resources. There are many ways to create and deploy a crawler trap.

2.1 Involuntarily crawler traps

Surprisingly, the notion of crawler trap exists from a long time [1], since some of them were not designed to be a crawler trap. This is what we call *involuntarily crawler trap*. For instance, for management purpose, we can find online interactive calendars which are constituted by an infinite number of pages, one for each month, generated on the fly by the webserver. Nowadays, most of calendars are developed in javascript but this language is prohibited on the TOR network for security reasons. This is why we can still find such objects online. Another example stands in some website which generate pages on the fly from a general link but with different parameters. Incorrectly managed by the bot, it is possible to make them endlessly loop on this type of link.

2.2 Infinite depth or infinite loop

In contrast, we can find intentional crawler which are designed for that purpose. The widely used solutions stands in the automatically generated pages when the system detects a bot. The content generated does not matter as long as the crawler hits the trap and that it crawls fake sub-pages. In the majority of cases, a loop with relative URL is created and the bot will crawl the same set of two pages. For instance, we can use two twin directories referencing each other such as the bots comes to one to go to the other and vice-versa:

```
http://www.example.com/foo/bar/foo/bar/
foo/bar/somepage.php
```

2.3 Identifiers

For some website, the strategy is to generate a unique identifier stored in the page URL. This one is assigned to each client visiting it in order to track him. This one is usually called a *session ID*. The bot — as any user — will receive such identifier. The trap lies in the fact that the user ID is changing randomly at random time. Such a way, the bot can be redirected to already visited webpages but with a different ID — an transparent action a regular user will not perform. The session ID is usually implemented in the URL as follows:

```
http://example.com/page?sessionId=3B959
30229709341E9D8D7C24510E383
http://example.com/page?sessionId=D27
E522CBFBFE72457F5479117E3B7D0
```

There are countermeasures to bypass such protection for bots, starting by registering hashes of already visited webpages to a smarter implementation of URL management.

2.4 Random texts

A different solution, which (in addition to its deployment) is more effective than others, stands in random generation of text in a webpage. With simple script from webserver side, one page can be randomly generated with random text and random links referencing the same page. Once this page is hit, the bot will treat what looks to it as *different* webpage, since it comes from different links with a different page's name, over and over.

But whatever is the level of sophistication of the crawler trap, it can only generate pages that respect the strategies for which it has been designed. It means that each crawler trap has specific characteristics. Either the web page is created automatically on the fly and the content is fully random, or it starts from an existing webpage whose content is randomly derived. In other words, the crawler can be working on totally fake platforms, where the pages are similar to the original site in terms of shape, HTML architecture, even some key words, part of content...

This is this last category of crawler trap which matters for us. Indeed, the involuntary one can be avoided by a bot aware of such objects. And about intentional ones, where they are based on URL management, it is possible to bypass the security with a specific management of already visited pages. But the last technique with random text generation cannot be managed in the same way and it requires a specific approach to handle it. This is the purpose of our paper.

From an operational point of view, our bot visits websites, page after page. The main difficulty lies in the fact that the trap generates random pages after the bot has visited a certain number of pages (based on the speed of the visit, the number of visits, the logic of the order of the links visited, etc). In a

way, we can only base our decisions on the already visited pages and the ones we are going to visit next. This is the operational context in which we operate. Our whole approach is not to avoid being detected by the website (crawling performance would be too downgraded) but to detect whenever the website starts setting a trap or not. For this reason, we need to measure the distance between regular and irregular webpages.

3 The use of distances

The objective is to measure the distance between several data sets to perform a distinction between several families. This can be done in Sect. 3.1 in order to define precisely what we are really trying to measure and the approach which drives our forthcoming analyses. Subsequently, we will be able to present some of the existing distances in the Sect. 3.2 and then we will discuss the contributions proposed by our distance to answer our problematic.

3.1 Approach to resolve the problem

From a generic point of view, we have two *ethnic groups*:

- Pages extracted from regular website ;
- Pages generated by crawler traps.

In each of the ethnic group, it is possible to find different *families* or subgroups. In the case of regular websites, it means that all the pages of a family come from the same website. The same applies for pages generated from different crawler traps.

In our case, we are trying to detect whenever there is a webpage generated from a crawler trap in a set of regular webpages or not. According to our operational context, we are aggregating webpages on the fly, which means we cannot know the full dataset in advance. It means we have to check the difference of a new webpage from a set of already visited webpages. This is doable by computing a distance $\mathcal{D}$ between the new webpage and the current set. In that sense, we can assume that a family F composed of n samples, $\forall i, j \in \mathbb{N}^{+*}$ such as $1 \leq i \leq j \leq n$ we have the random value X which models the distance between two samples s_i and s_j such as $X = \mathcal{D}(s_i, s_j)$. Since all samples belong to the same family, there is no reason that the distance between two samples is different to a third one, that is to say $\forall i, j, k \in \mathbb{N}^*$ where $i \neq j \neq k$ we have $\mathcal{D}(s_i, s_j) \approx \mathcal{D}(s_i, s_k)$. This is a consequence of the transitivity of a distance.

For optimization purposes, we cannot compute a distance between the new one and all pages of the current set. Since the distance between webpages of a single family is supposed to be the same, we can consider the mean (expected value)

as a good estimator. This mean is computed on the fly by updating it with each new value which belong to the family. It means that our detector system is based on the fact that the distance between a new sample s and the mean of a family m is based on the fact that $\mathcal{D}(s_i, s_j) \leq \epsilon$ where $\epsilon \leq 0$ is distinctive for each family.

The challenge is therefore to define a distance $\mathcal{D}$ that respects the property defined hereafter. It means that our distance must be *discriminating* in the sense that each family must have its own mean of distance. And it should be *accurate* which means that the standard deviation from the mean is supposed to be as small as possible.

3.2 Distances evaluated

As part of our research, we studied many distances. Among those selected, we kept three of them with the requested properties as described in Sect. 3.1: the Jensen-Shannon distance (JSD) [7], the Hellinger distance (HD) [8] and Bhattacharyya's distance [9,10]. Moreover we have designed many custom distances to solve our problem more efficiently.

For all distances, we consider the content of two webpages as random values X and Y composed of words. Each word (or group of words) in a page has a probability of occurrence. The set of all of these probabilities constitutes a probability distribution usually noted P and Q from the same space $\mathcal{X}$.

3.2.1 Jensen-Shannon

The *Jensen-Shannon divergence* (JSD) [7] is based on the idea that we should be able to balance the different distributions involved in the divergence, according to their importance. As for previous distributions, it aims at measuring the diversity (or the dissimilarity) between two populations. With $\pi_1, \pi_2 \geq 0$ real positive numbers such as $\pi_1 + \pi_2 = 1$ which are the *weights* of the two probability distributions, we define the Jensen-Shannon divergence as:

$$JS_{\pi}(P, Q) = H(\pi_1 P + \pi_2 Q) - \pi_1 H(P) - \pi_2 H(Q)$$

where H is the entropy function as defined by Shannon [11] on a random variable such as:

$$H(X) = - \sum_{x \in \mathcal{X}} p(x) \log_2 p(x)$$

In order to manipulate a true metric, it is common to compute the square root of the JSD divergence such as:

$$D_{JS_{\pi}}(p_1, p_2) = \sqrt{JS_{\pi}(p_1, p_2)}$$

3.2.2 Hellinger

The *Hellinger distance* (although known under the name of *Matusita measure* [8]) has been designed to check if one probability distribution follows another probability distribution and if it is a true metric [10]. The normalized version (the values are between 0 and 1) is as follows [12]:

$$\mathcal{D}_H(P, Q) = \frac{1}{\sqrt{2}} \sqrt{\sum_{i=1}^n (\sqrt{p_i} - \sqrt{q_i})^2}$$

3.2.3 Bhattacharyya

The Bhattacharyya measure for normal distributions is a very convenient equation to evaluate class separability. Bhattacharyya [9,13] defined a measure of distance between two multinomial populations, each consisting of n classes. These two populations are characterized by a set of probabilities $(p_1, \dots, p_n)$ and $(q_1, \dots, q_n)$ with $\sum p_i = 1$ and $\sum q_i = 1$. Bhattacharyya considered the difference of probabilities as the direction cosines of the two vectors in n -dimensional space referred to a system of orthogonal coordinates axes [14]. The measure used here is the square of the angle between the two straight lines through the origin. If Θ is the angle between the two lines, we can define the Bhattacharyya's distance as:

$$\cos \Theta = \sum_{i=1}^n \sqrt{p_i q_i}$$

Even if Bhattacharyya's distance has been used in many operational applications [15,16], it does not fulfill the requirements imposed by a metric structure since it does not respect the triangular inequality [17]. In order to improve this undesirable property, Comaniciu and al. [10] have defined a true metric distance between distributions based on the Bhattacharyya coefficient:

$$\mathcal{D}_C(P, Q) = \sqrt{1 - \rho(P, Q)}$$

where $\rho(P, Q)$ denotes the Bhattacharyya coefficient, although known as the *affinity coefficient*. The proof of the metric, provided in appendix of [10] is focused on the triangular inequality and proven by employing the half-angle sinus formula and a few trigonometric manipulations.

3.2.4 Custom distance—Shark

The main drawback of the existing distances we observed is that the random variables they consider are defined on the same space $\mathcal{X}$. It makes sense in simple cases but our one is a bit more specific. Indeed, we are computing the set of

words page after page and it means we do not have access to the set of all words. In a page, we can find words which are not present in the next page (and vice-versa) for the sake of example.

The main issue when we use existing distance is that, to be mathematically exact, we must remove unique words from each page. This can be explained by the way the probabilities in each set of words are computed. Technically speaking, the probability of occurrence is the number of occurrences of a word divided by the total number of words. But the divisor itself depends on what is considered to be the total number of words in the set. Either it is the total of words per set or the global total of words (conjunction of both sets). In the case where there is no unique word in on set, all mathematical properties of existing distances are independent from this fact. Otherwise, a bias can be inducted in the probability distributions, which means that some formulas will not be exact anymore. This is why such transform is justified. The possibility to preload all possible words from website is out of scope in our operational context since we compute the set of words, page after page, on the fly and considering unknown words on a page with a probability of zero does not fulfill all mathematical designs (we artificially increase the number of words and we change the probability distributions to remain consistent). Such a loss deprives us of information and therefore a loss of accuracy for our measurements. The idea of our measure has its roots in this simple observation.

The idea of one of our measures, called *Shark* is based on the rate of the entropy of the symmetric difference on the entropy of the two distributions. In other words, we look at the information contribution of the exclusive words from the two pages on all the information embedded in these two pages. Considering $P \in \mathcal{X}$ and $Q \in \mathcal{Y}$ we have $P \setminus Q = \{\forall x \in X \text{ where } x \notin Y\}$ we define our distance such as:

$$\mathcal{D}_s(P, Q) = \frac{H(P \setminus Q) + H(Q \setminus P)}{H(P) + H(Q)}$$

3.2.5 Custom distances—Archerfish and Handfish

In our case, we want to group different web pages that belongs to the same family. We can make the hypothesis that on a web site (but the observation could be used in a broader context) the percentage of differences between two linked pages follow a globalized deviation. In other word, the average percentage of change between two pages from the same website should not be so different from a page to another one. The idea which drove the conception of the distance was to minimize the average difference between two pages of the same family while maximizing the distance between two pages which are either too close or too different.

Arcsine distribution [18,19] is interesting to model a distance which would follow our requirements. In probability

theory, the arcsine distribution is known to have a cumulative distribution function such as $F(x) = \frac{2}{\pi} \arcsin(\sqrt{x})$ and a probability density function $\forall x \in [0, 1]$ with $f(x) = \frac{1}{\pi \sqrt{x(1-x)}}$. This is a rather paradoxical law, since the expectation is the least probable value and extreme values are the most probable [20]. But for this law, it is possible to design specific distances able to provide interesting results in our case. Starting from the distribution function, we propose *Archerfish distance* such as, from two distributions P and Q , $\forall p_i \in P$ and $q_i \in Q$ we have:

$$\mathcal{D}_a(P, Q) = \frac{4}{\pi^2} \sum_{i=1}^n \arcsin(\sqrt{p_i}) \arcsin(\sqrt{q_i})$$

From the density function, using the same distributions P and Q , we propose another distance called *Handfish*:

$$\mathcal{D}_h(P, Q) = \frac{1}{\pi} \left| \frac{1}{\sqrt{p_i(1-p_i)}} - \frac{1}{\sqrt{q_i(1-q_i)}} \right|$$

3.2.6 Custom distances—Dottyback and Dhufish

If distances based on arcsin distribution provide interesting results, it is possible to provide more efficient distances by the use of arctangent [21,22] function as a distribution. We are not using arctangent distribution as defined by [23,24] since this one is quite confidential and far to match our needs. Instead, we design two distances. One directly extracted from arctangent definition is called *Dottyback*. It is defined $\forall p_i \in P$ and $q_i \in Q$, then:

$$\mathcal{D}_o(P, Q) = \sum_{i=1}^n \sqrt{p_i} \arctan(\sqrt{p_i}) \sqrt{q_i} \arctan(\sqrt{q_i})$$

Another distance is based on the derivative of arctangent function. This one is called *Dhufish* and can be used to measure the distance between two distributions P and Q .

$$\mathcal{D}_u(P, Q) = \left| \frac{1}{1 + (p_i(1-p_i))} - \frac{1}{1 + (q_i(1-q_i))} \right|$$

4 Experiments

When we have conducted these experiments, we did not forget the main objective of detecting a crawler trap automatically. From an operational point of view, the crawler processes a website, page after page, and can fall into a crawler trap at any time. It is nearly impossible to detect a crawler trap with only two consecutive pages. The concept we developed is to train a classifier progressively with n

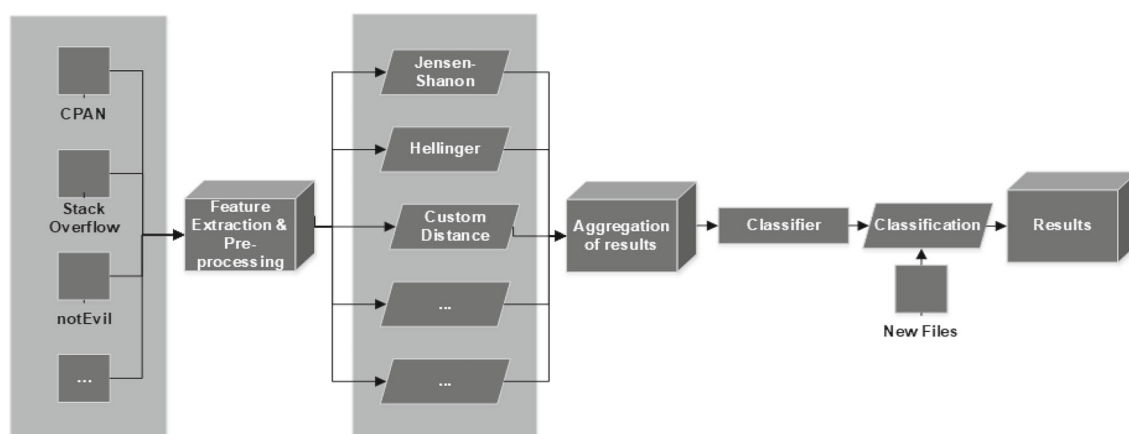


Fig. 1 Full process to determine a page family

consecutive pages then to detect whenever a page is too different or too similar from the previous ones. The process is schematized in Fig. 1. At the time, only the creation of a more discriminating distance is presented. The implementation, the choice of a classification algorithm and the operational tests will be the outcome of future works.

Since we have defined properties and the protocol to evaluate different distances, it is relevant to focus on which data we are going to process. First, we propose to explain which are the websites used to help our procedure to distinguish between regular websites (for what it could mean) from crawler traps. Then, we propose to explain which data are relevant in a web page so that it will help to build efficient detection system.

4.1 Experimental dataset

The approach we propose is requiring the presence of a dataset, which will be presented in the Sect. 4.1.1. More than explaining the context of our paper and the different actors involved, the idea is also to open our approach to make it reproducible and leave the results being critically analyzed, by anyone, as we did in Sect. 4.2.

4.1.1 Websites

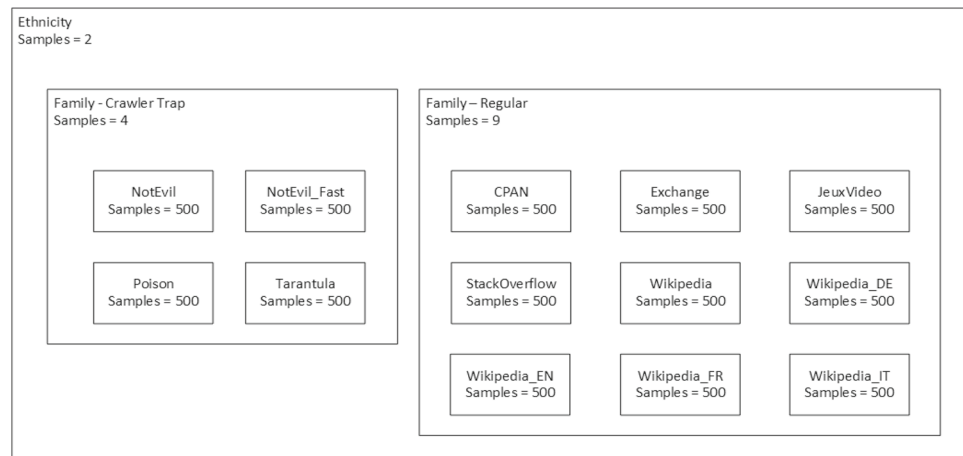
As there is no dataset available for our problematic — it has not been studied in the past —, we created our own. In this dataset, we chose to collect webpages from 13 distinct sources, each with different linguistic properties. To gather the dataset, we used a simplified version of our crawler. For some webpages like Wikipedia, we “clicked” on the random generation of article on the left side of the page (available in all languages) to have various panel of subjects. The inescapable component is that all pages are one click away

(from a bot point of view) from the previous and next one, hence they all are consecutive pages.

In the collected set, 4 sources are from a crawler trap and 9 are from “classic” websites. We collected 500 pages from each sources. There are divided in two main categories: regular websites and crawler trap ones. The first is about “classic” websites that anyone can easily find online. We wanted to have enough material from “spoken-language” to have relevant tests. Among the different websites evaluated for this purpose, we have selected the following detailed list (Fig. 2):

- **CPAN - 1 set (F1):** “*Comprehensible Perl Archive Network*” is a documentation for each Perl libraries. It can be a very long explanation or just few lines of codes. People wrote the documentation in a good English.
- **Jeuxvideo.com - 1 set (F2):** *JeuxVideo.com* is a well known French website for news and discussions about video games. A part of the website is a forum for teenagers with very few moderation. The language used is an approximate French, not well mastered but still understandable for native speakers.
- **Stack Exchange - 1 set (F3):** Network composed of 174 communities. The topics covered are various and range from science to religion through history, literature or mathematics. A proper English is mandatory for the validation of the question asked.
- **Stack Overflow - 1 set (F4):** One of the network in *Stack Exchange* specialized in computer programming questions. There is less English sentences and more sources codes as an example of their issues.
- **Wikipedia - 5 sets:** *Wikipedia* is the most known free encyclopedia. The language is very accurate. We have downloaded 5 sets, one by language (German (F5), English (F6), French (F7) and Italian (F8)) and the last

Fig. 2 Summary of data



one is composed of a combination of this four previous languages (F9).

The second category is about crawler-trap websites. We have to admit that it exists really few crawler-traps, at least identified. Why? Because most of the techniques used are based on simple values saved in cookies in web browsers. They are not hard to bypass for a trained enough bot. The real difficulties arise when the targeted website adapts the content of the webpage when it is close to detect a bot instead of a real human. To achieve such a goal, it exists few scripts or programs able to perform this task. More than a long list, the following one aims to represent all the diversity we have found about:

- **notEvil - 2 sets:** *notEvil* is originally a search engine in the TOR network. On this website, there is an embedded game named Zork. This game is associated with an identifier and, on the page, there are three links to a new game. Hence, a crawler trap is created because once the bot hits this link, it will play games indefinitely. On the same page, a small part is for “Recent Public Channels” and changes every time a channel is created or someone responds to an old one. To determine if this change has an impact on your classification, we collected two sets: one is composed of webpages downloaded (F10) as the bot does (everything is collected in the same minute) and one set (F11) is for a long time gathering process (one webpage per hour). This changing part is usually spambots, URL links or unethical topics.
- **Poison - 1 set (F12):** *Poison* [25] is the most random crawler trap we ever met. There is no practical example available online, therefore we have downloaded the software (developed in 2003) and generated our sample of 500 pages. Through this, we had the possibility to play with several coefficients to generate a real random sample of pages:

- Number of paragraphs [1:10]
- A CSS background (adding HTML tags and data to create a colorful page) [0:1]
- Minimum number of words per paragraph [25:75]
- Number of words added to the minimum number of words per paragraph [25:100]

This crawler trap is based on the operating system dictionary, thus, a lot of words are uncommon and very advanced, even for native speakers (ex:*thionthiotic*). In this system, the probability of picking up the word “house” is the same as the word “chorioretinitis”.

- **Tarantula - 1 set (F13):** This is the name of a PHP crawler trap available on GitHub [26]. It generates a simple page with 1 to 10 new links and a text that contains between 100 and 999 words chosen randomly in a list of 1000 predefined words. *Tarantula* is not online so we have configured it on our local web server and generated 250 samples from the root address and 250 by following a random link of the page.

In order to ensure *reproducibility* [27] of results presented here, the dataset (514.3 MiB uncompressed) will be made available on <https://github.com/MaxenceD-ConfData/ForSe-2020>.

4.1.2 Relevant data in a webpage

There is no defined rules for a functioning crawler trap. It can be located on a small part of the webpage just as it can be on the whole page. And, as explained previously in Sect. 1, a web page is composed of different parts. A crawler trap can have an impact on all of these different parts. The shape of the page (more precisely the way or the style in which it is written) and the content (words which are displayed to user’s eyes) are equally relevant to classify. Therefore, we divided the page in three parts and created four main features:

- A list of words contains in the `<body>` tag of the page. To collect them, we used the following regex: `m/ w+/. We convert all words in lower case to normalized data.`
- The HTML tag architecture starting at the `<body>` tag. To remove all text between each tag, we used the regex `s/> [\^ <]+ < / >< /`. We also removed comments and links found in `<src>` and `<href>` tags.
- The metadata of the page located in the `<head>`. Comments and text between tags were removed.
- For comparison purposes, the fourth category is composed of the list of words and the HTML architecture. The same pre-processing were applied for each part.

From a formal detection point of view, HTML tags provide rather efficient results. This is because the design of a web page is rarely a copy of an existing one. Some changes have been performed in the general shape but in addition to the number of paragraph, pictures, links and so on present on the page. But, to be used in a context where the design would not matter or where it does not exist by itself data which is different enough to be easily detected, we have chosen to focus on the content of a web page. For the sake of universalism of our detection method, we have privileged the content of a page as the *de facto* data where to perform the detection. It helps to generalize our process for other data in other contexts. This is why results provided in Sect. 4.3 are focused on web page's content. But first, we need to introduce in Sect. 4.2 how we perform the evaluation of our distances compared to existing ones.

4.2 Experimental evaluation

Since the content of the material used for experiment has been described, it matters to evaluate our distances comparing to existing ones. The proposed method in the following has been applied to all distances in this paper, including ones and our own ones. Cross-validation [28] is an usual method to evaluate the efficiency of a data-mining detection model. But if it is efficient, it does not give information about potential flaws of the method evaluated. In addition, it is designed to perform an evaluation on a fixed set of data. However, our method is designed to be *dynamic* in the way it builds its dataset and on the fly it performs the detection. Indeed, we need a measure which makes sense when the population evolve in the time when the observation is performed. Since pages are collected on the fly, at a given time t_i statistical parameters of population $\text{Pop}(\mu, \sigma)$ can be different at time t_{i+1} with $\text{Pop}(\mu', \sigma')$. It would not be realistic to perform cross-validation method for each new page collected by our bot and even if, what mean all these results together would.

Solution we propose is simpler and it returns to the fundamentals of what a statistical detection test (as defined by Fisher [29]) is. Indeed, from a population P composed

of families F_1 and F_2 , we draw samples E_i . The output $X = \mathcal{D}(x, y)$ of distance $\mathcal{D}$ where $x, y \in E_i$ can be considered as the output of random variables X and Y . Thus, we can define the mean of the distance $E[X] = E[\mathcal{D}(X, Y)]$ and the standard deviation $\sigma_{\mathcal{D}}$. In family F_0 , $\forall x, y \in F_0$ and $\forall t \in \mathbb{R}$, we expect with a certain confidence that the any element in F_0 are clustered with respect to the distance $\mathcal{D}$ with

$$\underbrace{E[\mathcal{D}(x, y)] - t\sigma_{\mathcal{D}} \leq \mathcal{D}(x, y) \leq E[\mathcal{D}(x, y)] + t\sigma_{\mathcal{D}}}_{ID_0}$$

where ID_0 represent the ideal confidence interval for distance $\mathcal{D}$ with family F_0 . More directly, $\mathcal{D}(x, y) \notin ID_0$ means that x and y belongs to different families.

Taking into account that we are dealing with two families defined by their ideal confidence intervals ID_0 (regular web pages) and ID_1 (crawler traps), knowing that a page belongs to one population or another is equal to perform a statistical hypothesis test. Indeed, we can compute ID_0 experimentally with a large enough samples set such as $E[\mathcal{D}(x, y)] = \mu'_0$ where $\mu'_0 \xrightarrow{P} \mathcal{L}(\mu_0, \sigma_0)$ and μ_0 and σ_0 represent the real statistical moments for a given population following the statistical law $\mathcal{L}$. The same way with ID_1 , $E[\mathcal{D}(x, y)] = \mu'_1 > \mu'_0$ and $\mu'_1 \xrightarrow{P} \mathcal{L}(\mu_1, \sigma_1)$. The goal is to get a μ_0 which is different from μ_1 so that distance $\mathcal{D}$ makes possible the distinction between the two families.

Considering the average of the distances as the sum of normalized random variables, it becomes possible to apply the consequences of the central limit theorem [30,31] which says that the resulting variable (average of distances, in our case) tends towards the normal distribution. It implies that $\mu'_i \xrightarrow{P} \mathcal{N}(\mu_i, \sigma_i)$. By consequence, according to normal distribution properties, the interval $[\mu - \sigma; \mu + \sigma]$ should contain 68 % of the observed distances applied to elements from a given family. More generally, we can consider the following intervals where we increase the confidence factor value t applied on the standard deviation:

- $[\mu - 1.96\sigma; \mu + 1.96\sigma]$ should contain 95 % of observations ;
- $[\mu - 3\sigma; \mu + 3\sigma]$ should contain 99 % of observations.

Of course, there is a possibility that it exists an overlap between two intervals. If there is an *area of interference*, there is a possibility of misdetection. The goal of our designed distances is to reduce this overlapping so that each family has its own area without being covered by another. For short, the best distance is the one where the area of interference between ID_0 and ID_1 is the smallest.

In literature, it exists methods to make comparison of means defined with confidence intervals [32–34] with restric-

tions to how they should be used [35,36]. Unfortunately such methods are usually focused on determining a null hypothesis affiliation test between two samples (or to a reference population) more than measuring the efficiency of the statistic or the distance used. To proceed, we propose to calculate the overlap rate of confidence intervals. It means that in the case it should be possible to draw all the confidence intervals on the same line, we could compute the ratio between space shared between at least two confidence intervals and the total space covering by all confidence intervals.

There are very few papers about measuring the covered space of overlapping intervals. One relevant is [37] which defines a novel similarity measure for interval-valued data based on overlapping ratio. Their measure is based on the Jaccard similarity measure [38] and on the Dice similarity [39,40] — the two most used similarity measurement indices. To proceed, they design their own similarity measure $S(A, B) \rightarrow [0, 1]$ as a function that determines the similarity between two objects A and B . *De facto*, they are measuring a ratio based on the overlapping rate (if $A \cap B$ then $S(A, B) > 0$ otherwise $S(A, B) = 0$) between intervals. They define four common properties of a similarity measure [41]:

- Boundedness: $0 \leq S(A, B) \leq 1$;
- Symmetry: $S(A, B) = S(B, A)$;
- Reflexivity: $S(A, B) = 1 \iff A = B$;
- Transitivity: If $A \subseteq B \subseteq C$ then $S(A, B) \geq S(A, C)$.

If Jaccard and Dice similarity indices are relevant, they miss two important points which are relevant in our case. The first is that these measures are used only on two intervals (or set of intervals) at a time. It means we have always a distinctions between two families. However, our evaluation deals with more than two families to be realistic. In addition, intervals handled by existing indices have fixed bounds. However, we act in the case of confidence intervals where the confidence level t on the standard deviation may vary. Such a property helps us to measure the robustness of our measurement at various levels of precision. But it implies a more extensive problem of transitivity as defined in [41]. Indeed, we have to deal with the *non-negative monotony* of the similarity such as: $\forall t, t' \in \mathbb{R}$ with $t \leq t'$ then $S(I_i, I_j, t) \leq S(I_i, I_j, t')$

Solution we propose respects the properties expected in [41] in addition to respect the non-negative monotony we introduce. Technically, this last property is just a linear extension of the transitivity property proposed before. The index we propose is designed to keep the main idea of Jaccard index but taking into account multiple overlapping of several intervals in addition to keep the non-negative monotony. From a set of n intervals, where each interval can be written such as

$I_i = [a_i, b_i]$, we define the *overlapping rate* as $\forall i, j$ with $1 \leq i < j \leq n$:

$$\begin{aligned} \tau &= \sum_{i=1}^n \sum_{j=i+1}^n c_n \frac{\Delta(\{I_i \cap I_j\})}{\Delta(\{I_i \cup I_j\})} \\ &= \frac{2}{n(n+1)} \sum_{i=1}^n \sum_{j=i+1}^n \begin{cases} \frac{\text{Med}(a_i, b_i, b_j) - \text{Med}(a_i, b_i, a_j)}{\text{Max}(b_i, b_j) - \text{Min}(a_i, a_j)} \\ \text{if } \text{Max}(b_i, b_j) - \text{Min}(a_i, a_j) \neq 0 \\ 1 \text{ otherwise} \end{cases} \end{aligned}$$

where Med corresponds to the median function. This one is used in the numerator to compute the common size shared by two intervals (when they are overlapping).

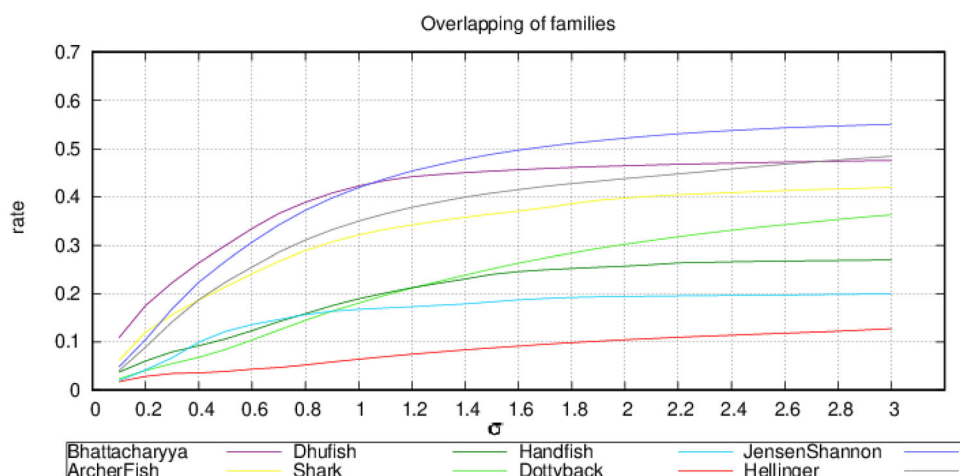
4.3 Experimental results

Results provided in Fig. 3 are given for the overlapping of confidence intervals computed for each of the distances presented here. This one is generated from distances driven by the content text extracted from web page. Focusing on the content is that it provides more results which can be applied beyond to the context of crawler-trap detection. To get accurate results, we decided to consider measures on an input set manipulated in bi-grams with overlapping. It means that probability distributions taken into account by evaluated distances are built from *words* joined together two by two (with an overlap so that the same word can be used in two joints). This computation of the overlapping of confidence intervals in Fig. 3 is given from family to family. In this case, it does not matter that a website is considered to be a regular website or a crawler-trap. Each website is considered by itself and we check if the different distances are able to transcribe a clear difference between each other.

On Fig. 3, the overlapping rate of the different confidence intervals is given on the y-axis. This one goes from 0 to 1 to represent an overlap rate of 0 to 100 %. On the x-axis is represented the confidence coefficient t . It is the one that is used as a confident coefficient for confidence intervals with $[\mu - t\sigma; \mu + t\sigma]$. Such construction makes possible to artificially increase the length of the intervals to evaluate our distances under extreme conditions. Plots on the chart represent the evolution of the overlapping rate when t is increasing, as explained in Sect. 4.2. Technically, the more efficient the distance is (which means the better the detection with the distance is), closer to the x-axis the plot modeling of the distance's evaluation must be.

We can see that the-state-of-the-art distances are given the worst results. Indeed, their plots are always above the ones of our distances, whatever the are the state-of-the-art distances. A special mention to *Handfish* and *Dottyback* distances which have the lowest rate of overlapping. With 19.95 % and 12.73 % of overlapping when $t = 3$, the performance is remarkable (compared to the 47.61 %, 48.48 %

Fig. 3 Overlapping rates computed from families to families. X-axis gives the confident coefficient t (for $[\mu \pm t\sigma]$) and y-axis gives the overlapping rate of confidence intervals generated by distances



and 55.11 % for respectively Bhattacharyya, Hellinger and Jensen-Shannon distances). These results are not so bad when we consider that objects compared are both very similar in their nature (websites) but very different by themselves since web pages of a single website are far to all be similar. By consequence, in practice, it guarantees a strong detection efficiency with a very low rate of false positive for the large majority of observations in the sample.

If we are taken into account the notion of clusters (regular web pages and trapped ones), the results are all the more cruel for distances from the state of the art. Indeed, Fig. 4 provides results with the notion of clusters taken into account. It means we have reused our similarity index defined in Sect. 4.2 but the two sets provided have been first merged with the different families which compose each cluster (regular and trapped web sites). This construction of merged intervals can impact the monotonicity of the overlapping rate since intervals are technically merged more than once. But the comparability of the results remains the same since the proportions are kept. The output of this merging operation is still a set of confidence intervals where it is possible to compute our similarity index between the two.

As given in Fig. 3, results in Fig. 4 still confirm the superiority of our distances compared to those of the state of the art. The efficiency of our distances is as important as Dotyback is in a near perfect detection situation if not a possible and infinitesimal residue of false negatives (0.57 % when $t = 3$). For the sake of simplicity, such a result means that our distance is about to almost always detect observations as in a cluster or another with almost no error in more than 99 % of the time, according to Sect. 4.2.

To evaluate the impact induced by distance for each family from a cluster point of view, it is possible to reuse our similarity index, but this time we do not merge the confidence intervals of families in clusters anymore. The result is given in Fig. 5. Here we consider the overlapping rate induced by the interval for each family in the first cluster

reported on the intervals of families in the second cluster by a given distance. In the case of results given in Fig. 5, we measure the impact of regular websites on trapped ones. This time, since there is no merging manipulation on input intervals, the rates' plot are monotonic. It remains that Dotyback has still good results (impact of overlapping is 7.28 %) and Shark has 3.4 % of overlapping impact rate, which is better. Hellinger has an impact of 17.31 %, Bhattacharyya 20.44 % and Jensen-Shannon 28.08 which confirms that state of the art distances are the distances which induce the most overlapping. Such impact index helps to predict bad classification by a distance from a cluster to another. The lower it is, better the detection is.

5 Detection model to use

Once the distance has been selected, we had to pay attention to the detection model. In machine learning, there are two main sorts of detection model that can classify information [42]. The first one is called *supervised learning* [43]. Useful if a large part of the work can be done beforehand, especially in the aggregation of data to define what to detect. However, in the case of crawler traps, it means that we need to train the model with a large training set composed of different pages from regular websites and crawler traps. But it is not possible to collect by advance all different pages of regular and trapped websites.

Unsupervised machine learning [44,45], the model is crafted on the fly, object after object. If the unsupervised learning does not require prior work, it still work in two phases: (learning and interpretation (clustering in our case)). Once the model is created in the first phase, the second consists of classifying the information it contains. To proceed, the model will be clustered into different groups (where the number is predefined for use $k = 2$ since we perform the detection between regular and trapped websites). Each clus-

Fig. 4 Overlapping rates computed from the two clusters. Distances are still evaluated but inputs is two sets of intervals representing the two clusters (regular and trapped websites) where confidences intervals have been merged

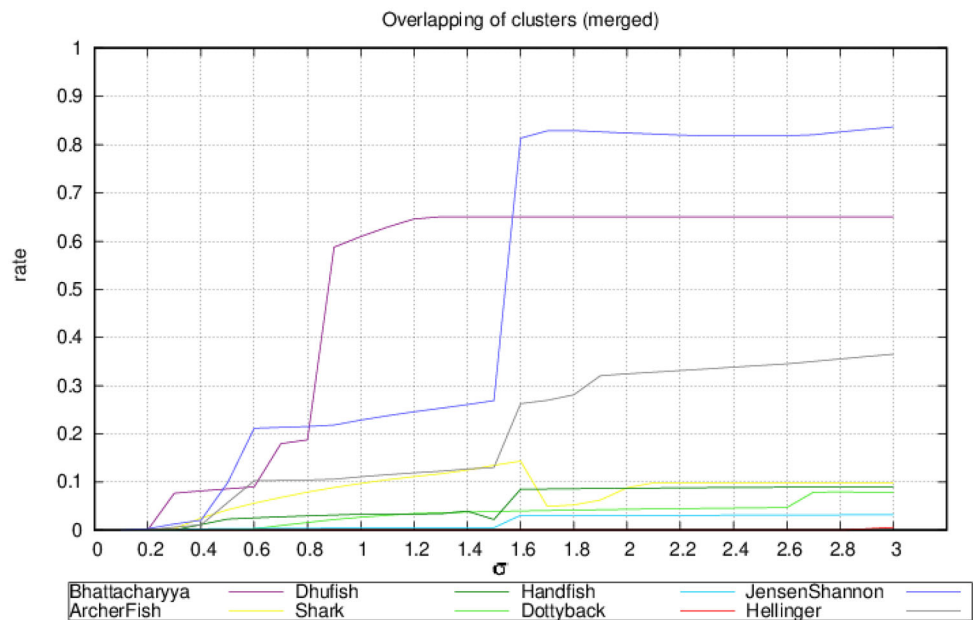
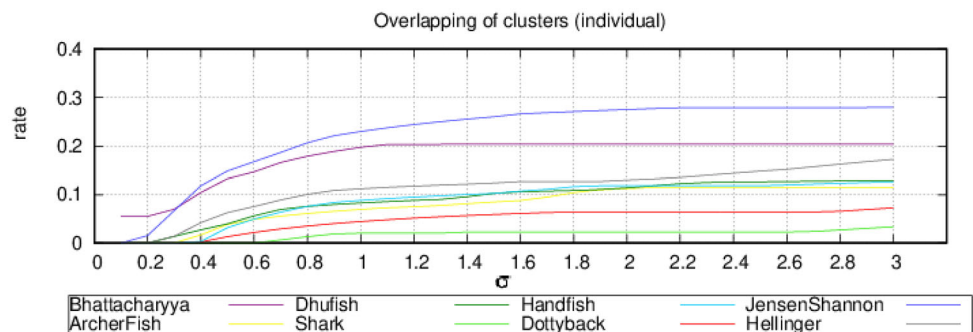


Fig. 5 Overlapping rates computed from the two clusters by family. It helps to measure the impact on badly detect a cluster by a given distance



ter is created by determining a central point around which the cluster will be defined. In most of machine learning clustering algorithm, the number of core points (clusters) can be defined by the user and they are randomly tested [46] to find the best classification. But in our case, the pre-training phase is hard to compute. First because when we are starting to evaluate a website with a bot, we do not have large set of regular pages to constitute a reference set. Then, because web-pages in a website can be very different from one to another, meaning that there are some atypical observations collected by the bot. Such pollution of the set can have a strong impact in our detection which, one more time, is designed to be trained on the smallest possible set. In practice, a number of 10 web pages is enough for the regular web site.

From these constraints, we have created our own dynamic classification model mainly based on unsupervised machine learning and robust statistic classification [47,48]. The main breakthrough on this model lies in the robustness and the dynamic part. The dynamic part lies in the fact that our model of detection is crafted on the fly. Starting from the entry page of a website, we design what is the website from page with

our bot, page after page. This process generates the different clusters by using the *silhouette method* [49]. The Silhouette method consists in computing the average distance between the new object and all points of the cluster. This new object will be included in nearest cluster. The need of a dynamic model can also be expressed on a larger scale. Moreover, our detection model is designed to adapt to the case where the data is moving during the measurement campaign. Measuring the size of people in a population can also be an example. Indeed, the size of the individuals may have varied between the time the measurement campaign is started and the time the result is given. The second part is the robustness of the model. Indeed, by removing outlier data [50] on the fly, it is possible to build a more accurate cluster model and therefore a more reliable detection model.

To present our detection model, let us start by considering $X = (X_1, \dots, X_p)^t \in \mathbb{R}^p$ a p -dimension random vector driven by a distribution F and $x = \{x_1, \dots, x_n\}$ with $x_i \in \mathbb{R}^p$ a n -size random sample taken from the random vector X . We draw the assumption that every x_i in the sample x does not lie

in less than k points to be coherent with our analysis (where k is the number of clusters considered — 2 in our case).

The use of k -means to classify objects is maybe the most widely used classification method. This method aims to create clusters by minimizing the sum of the squares of the distances between each observation and the center of the nearest cluster. The first formalized articles about this method are from Dalenius [51], Cox [52] and Fisher [53] but others authors used it before in different contexts or shapes. Result of this method is given as a set of k point centers $\{T_1, \dots, T_k\} \subset \mathbb{R}^p$. Technically, the k -means set $\{T_1, \dots, T_k\}$ is defined by solving the following minimization problem:

$$\min_{T_1, \dots, T_k} \sum_{i=1}^n \left(\inf_{1 \leq j \leq k} \|x_i - T_j\|^2 \right)$$

As it happens with the sample mean (i.e., the 1-mean), the k -means method suffers from a serious lack of robustness trouble [54]. Indeed, the k -means method is highly susceptible to variations in the sample. To avoid issues link to aberrant or atypical observations (outlier points), we can propose solution to use k -means method in a more robust way. García-Escudero *et al.* [55] provided several propositions where we used “trimmed k -means”. The objective is to use k -means methods but trimming a rate of atypical observations. This is done by removing from the sample observations detected by a distance in regard to the center of observations. But as explained in [56], in some cases, outlier points are not so far from the center of different clusters and a distance is no more reliable to detect them. Gordaliza [57] proposed that observations to remove do not depend on arbitrary decisions but can be defined directly from data. They call this concept *impartial trimming*. With this concept, Cuesta-Albertos *et al.* [58] introduced the concept of trimmed k -means.

From a formal point of view, trimmed k -means $\{T_1, \dots, T_k\} \subset \mathbb{R}^p$ minimize the trimmed sum of the the squares of the Euclidean distances inside clusters such as:

$$\min_{x_{n\alpha}} \min_{T_1, \dots, T_k} \sum_{x_i \in x_{n\alpha}} \inf_{1 \leq j \leq k} \|x_i - T_j\|^2$$

where $x_{n\alpha}$ is a subset of x holding $n - \lfloor n\alpha \rfloor$ observations. Another method we can use to improve our results (and obtaining clusters with an ellipsoidal shape) is TCLUST procedure [59,60].

6 Conclusion

The present paper is an extended version of our ForSE 2020 paper [61]. This version provides more distances and details about then. In addition, our original method of confidence

intervals overlapping has been given to explain and prove the strength of distances used to solve our problem. Finally, we provide a dynamic protocol to perform detection by the use of our distances and with the conjunction of robust statistics to remove aberrant observations from our training set crafted on the fly.

From this work, we have the capacity to decide with accuracy whether a web page belongs to which family (crawler trap or not) with better results compared to existing information distances. Our results are characterized with a dedicated means for different family and lower standard deviation compared to existing distances. Such result mean less overlapping between families and finally a better result in object clustering.

Since we can detect that we are about to be trapped by the targeted website, the ability to avoid it can be computed not so hardly. For the sake of simplicity, it would require checking a subset of previously visited websites and avoid to go on pages with a distance too far from that set. Consequently, we can focus on relevant links and avoid trapped ones.

Finally, our results can be used in a wider context than just web-pages classification. Indeed, our approach is generic enough to be applied in any statistical problem or clustering issue with much better results than already existing distances in the state of the art. In addition, the definition of a new index to evaluate overlapping of multiple confident intervals with an increasing coefficient of confidence is a very efficient method which help to design efficiently and quickly new distances by using a small quantity of computation tests. Moreover, our solution fits perfectly to any problem of dynamic statistics when the models are crafted on the fly by detection algorithm with a very subset of initialization data.

References

1. Steven, V.V., Ondrej, K., Vojtech, Z.: Crawler traps: how to identify and avoid them. <https://www.contentkingapp.com/academy/crawler-traps/> (2020). Accessed 21 Nov 2019
2. Rouse, M.: Chatbot definition. <https://searchcustomerexperience.techtarget.com/definition/chatbot> (2019). Accessed 21 Nov 2019
3. Google: Googlebot definition. <https://support.google.com/webmasters/answer/182072?hl=en> (2019). Accessed 21 Nov 2019
4. Hasan, W.: A survey of current research on captcha. *Int. J. Comput. Sci. Eng. Surv.* **7**, 1–21 (2016). <https://www.blackhat.com/docs/asia-16/materials/asia-16-Sivakorn-Im-Not-a-Human-Breaking-the-Google-reCAPTCHA-wp.pdf> (2016). Accessed 27 Mar 2021
5. Sivakorn, S., Polakis, J., Keromytis, A.D.: I’m not a human: breaking the google recaptcha. <https://www.blackhat.com/docs/asia-16/materials/asia-16-Sivakorn-Im-Not-a-Human-Breaking-the-Google-reCAPTCHA-wp.pdf> (2016). Accessed 27 Mar 2021
6. Techopedia: Spider trap. <https://www.techopedia.com/definition/5197/spider-trap> (2017). Accessed 21 Nov 2019
7. Lin, J.: Divergence measures based on the Shannon entropy. *IEEE Trans. Inf. Theory* **37**, 145–151 (1991)

8. Matusita, K.: Decision rules, based on the distance, for problems of fit, two samples, and estimation. *Ann. Math. Stat.* **26**, 12 (1955)
9. Bhattacharyya, A.: On a measure of divergence between two multinomial populations. *Sankhya Indian J. Stat.* **7**(4), 401–406 (1946)
10. Comaniciu, D., Ramesh, V., Meer, P.: Kernel-based object tracking. *IEEE Trans. Pattern Anal. Mach. Intell.* **25**(5), 564–577 (2003)
11. Shannon, C.E.: A mathematical theory of communication. *Bell Syst. Tech. J.* **27**(3), 379–423 (1948).
12. Nikulin, M.: Hellinger distance, *Encyclopedia of Mathematics*. http://www.encyclopediaofmath.org/index.php?title=Hellinger_distance&oldid=16453 (2011). Accessed 21 Nov 2019
13. Bhattacharyya, A.: On a measure of divergence between two statistical populations defined by their probability distributions. *Bull. Calcutta Math. Soc.* **35**, 99–109 (1943)
14. Aherne, F.J., Thacker, N.A., Rockett, P.I.: The Bhattacharyya metric as an absolute similarity measure for frequency coded data. *Kybernetika* **34**, 363–368 (1998)
15. Choi, E., Lee, C.: Feature extraction based on the Bhattacharyya distance. *Pattern Recogn.* **36**, 1703–1709 (2003)
16. Mohammadi, A., Plataniotis, K.: Improper complex-valued Bhattacharyya distance. *IEEE Trans. Neural Netw. Learn. Syst.* **27**(5), 1049–1064 (2015)
17. Fukunaga, K.: Introduction to Statistical Pattern Recognition, 2nd edn. Academic Press Professional, Inc., San Diego (1990)
18. Feller, W.: An Introduction to Probability Theory and Its Applications, vol. 1–2. Wiley, Hoboken (1968)
19. Kendall, M., Stuart, A.: The Advanced Theory of Statistics. Distribution Theory, vol. 1, 4th edn. Macmillan, New York (1977)
20. Saporta, G.: Probabilités, analyse des données et statistique, 2e édition révisée et augmentée. Technip (2006)
21. Zwillinger, D.: CRC Standard Mathematical Tables and Formulae: Ser. Mathematical Science References, 30th edn. CRC-Press, Boca Raton (1995)
22. Abramowitz, M., Stegun, I.A.: Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables, ninth dover printing, tenth gpo printing ed. Dover, New York (1964)
23. Pollastri, A., Tornaghi, F.: Some properties of the arctangent distribution. https://www.researchgate.net/profile/Angiola_Pollastri/publication/242525426_Some_properties_of_the_Arctangent_Distribution/links/551be9890cf2fe6cbf75fc42/Some-properties-of-the-Arctangent-Distribution.pdf (2004). Accessed 21 Nov 2019
24. Zenga, M.: L'impiego della funzione arcotangente incompleta nello studio della distribuzione asintotica dello scarto standardizzato assoluto massimo di una trinomia. *Statistica* **12**(XXXIX), 269–286 (1979)
25. Sugarplum – spam poison. <http://www.devin.com/sugarplum/>. Accessed 21 Nov 2019
26. Stickano/tarantula: Spider trap. <https://github.com/Stickano/Tarantula>. Accessed 21 Nov 2019
27. Nosek, B., Alter, G., Banks, G., Borsboom, D., Bowman, S., Breckler, S., Buck, S., Chambers, C., Chin, G., Christensen, G., Contestabile, M., Dafoe, A., Eich, E., Freese, J., Glennerster, R., Goroff, D., Green, D., Hesse, B., Humphreys, M., Yarkoni, T.: Promoting an open research culture. *Science (New York, N.Y.)* **348**, 1422–1425 (2015)
28. Stone, M.: Cross-validatory choice and assessment of statistical predictions. *J. R. Stat. Soc. Ser. B (Methodol.)* **36**(2), 111–133 (1974). <https://doi.org/10.1111/j.2517-6161.1974.tb00994.x>
29. Fisher, S.R.: 272: The nature of probability. *Centen. Rev.* **2**, 261–274 (1958)
30. Fischer, H.: A history of the central limit theorem. From classical to modern probability theory. <http://www.medicine.mcgill.ca/epidemiology/hanley/bios601/GaussianModel/HistoryCentralLimitTheorem.pdf> (2011). Accessed 21 Nov 2019
31. Pólya, G.: über den zentralen grenzwertsatz der wahrscheinlichkeitsrechnung und das momentenproblem. *Mathematische Zeitschrift* **8**, 171–181 (1920)
32. Knol, M.J., Pestman, W.R., Grobbee, D.E.: The (mis)use of overlap of confidence intervals to assess effect modification. *Eur. J. Epidemiol.* **26**(4), 253–254 (2011)
33. Payton, M.E., Greenstone, M.H., Schenker, N.: Overlapping confidence intervals or standard error intervals: what do they mean in terms of statistical significance? *J. Insect Sci.* **3**, 34 (2003)
34. Frost, J.: Using confidence intervals to compare means. <https://statisticsbyjim.com/hypothesis-testing/confidence-intervals-compare-means/> (2019). Accessed 21 Nov 2019
35. Austin, P., Hux, J.: A brief note on overlapping confidence intervals. *J. Vasc. Surg.* **36**, 194–195 (2002)
36. Parasurama, P.: Why overlapping confidence intervals mean nothing about statistical significance. <https://towardsdatascience.com/why-overlapping-confidence-intervals-mean-nothing-about-statistical-significance-48360559900a> (2017). Accessed 21 Nov 2019
37. Kabir, S., Wagner, C., Havens, T., Anderson, D., Aickelin, Novel U.: Similarity measure for interval-valued data based on overlapping ratio. https://www.researchgate.net/profile/Uwe_Aickelin/publication/318311193_Novel_similarity_measure_for_interval-valued_data_based_on_overlapping_ratio/links/5d0c6f15299bf1547c716709/Novel-similarity-measure-for-interval-valued-data-based-on-overlapping-ratio.pdf (2017). Accessed 21 Nov 2019
38. Jaccard, P.: Distribution de la flore alpine dans le bassin des dranses et dans quelques régions voisines. *Bulletin de la Societe Vaudoise des Sciences Naturelles* **37**, 241–272 (1901)
39. Sørensen, T.: A Method of Establishing Groups of Equal Amplitude in Plant Sociology Based on Similarity of Species Content and Its Application to Analyses of the Vegetation on Danish Commons, ser. Biologiske skrifter. I kommission hos E. Munksgaard. <http://www.royalacademy.dk/Publications/High/295S%C3%B8rensen,%20Thorvald.pdf> (1948). Accessed 21 Nov 2019
40. Dice, L.R.: Measures of the amount of ecologic association between species. *Ecology* **26**, 297–302 (1945)
41. McCulloch, J., Wagner, C., Aickelin, U.: Analysing fuzzy sets through combining measures of similarity and distance. In: IEEE International Conference on Fuzzy Systems. https://www.researchgate.net/profile/Uwe_Aickelin/publication/265337448_Analysing_Fuzzy_Sets_Through_Combining_Measures_of_Similarity_and_Distance/links/541adedf0cf2218008bfe73c/Analysing-Fuzzy-Sets-Through-Combining-Measures-of-Similarity-and-Distance.pdf (2014). Accessed 21 Nov 2019
42. Mohri, M., Rostamizadeh, A., Talwalkar, A.: Foundations of Machine Learning. The MIT Press, Cambridge (2012)
43. Aggarwal, C.C.: Data Mining: The Textbook. Springer, Cham (2015)
44. Khanam, M., Mahboob, T., Imtiaz, W., Ghafoor, H., Sehar, R.: A survey on unsupervised machine learning algorithms for automation, classification and maintenance. *Int. J. Comput. Appl.* **119**, 34–39 (2015)
45. Usama, M., Qadir, J., Raza, A., Arif, H., Yau, K.-L., Elkhatab, Y., Hussain, A., Al-Fuqaha, A.: Unsupervised machine learning for networking: Techniques, applications and research challenges, IEEE Access (2017)
46. MacQueen, J.: Some methods for classification and analysis of multivariate observations. In: Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, Volume 1: Statistics. Berkeley, Calif.: University of California Press, pp. 281–297. <https://projecteuclid.org/euclid.bsmsp/1200512992> (1967)
47. Huber, P., Ronchetti, E.: Robust Statistics, ser. Wiley Series in Probability and Statistics. Wiley, Hoboken (2011)

48. Herwindiati, D.E., Djauhari, M.A., Jaupi, L.: Robust statistics for classification of remote sensing data. In: 20th International Conference on Computational Statistics. COMPSTAT 2012, Limassol, Cyprus, pp. 317–328, proceedings of COMPSTAT 2012 ISBN:978-90-73592-32-2p. 317-328. [Online]. Available: <https://hal.archives-ouvertes.fr/hal-02468060> (2012)
49. Rousseeuw, P.J.: Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. *J. Comput. Appl. Math.* **20**, 53–65 (1987)
50. Aggarwal, C.C.: *Outlier Analysis*. Springer, Berlin (2013)
51. Dalenius, T.: The problem of optimum stratification. *Scand. Actuarial J.* **1950**(3–4), 203–213 (1950). <https://doi.org/10.1080/03461238.1950.10432042>
52. Cox, D.R.: Note on grouping. *J. Am. Stat. Assoc.* **52**(280), 543–547 (1957). <https://doi.org/10.1080/01621459.1957.10501411>
53. Fisher, W.D.: On grouping for maximum homogeneity. *J. Am. Stat. Assoc.* **53**(284), 789–798 (1958)
54. García-Escudero, L., Gordaliza, A.: Robustness properties of k means and trimmed k means. *J. Am. Stat. Assoc.* **94**, 956–969 (1999)
55. García-Escudero, L., Gordaliza, A., Matrán, C., Mayo, A.: A review of robust clustering methods. *Adv. Data Anal. Classif.* **4**, 89–109 (2010)
56. Droesbeke, J.-J., Saporta, G., Thomas-Agnan, C.: *Méthodes robustes en statistique*, Technip, Ed. <https://hal.archives-ouvertes.fr/hal-01126519> (2015)
57. Gordaliza, A.: Best approximations to random variables based on trimming procedures. *J. Approx. Theory* **64**(2), 162–180 (1991)
58. Cuesta-Albertos, J., Gordaliza, A., Matrán, C.: Trimmed k-means: an attempt to robustify quantizers. *Ann. Stat.* **25**, 553–576 (1997)
59. García-Escudero, L., Gordaliza, A., Matrán, C., Mayo, A.: A general trimming approach to robust cluster analysis. *Ann. Stat.* **36**, 1324–1345 (2008)
60. Adnan, R., Jedi, A.: Tclust?: trimming approach of robust clustering method. *Malays. J. Fund. Sci.* **8**, 253–258 (2012)
61. David, B., Delong, M., Filiol, E.: Detection of crawler traps: formalization and implementation defeating protection on internet and on the TOR network. In: 4th International Workshop on FORMal methods for Security Engineering (ForSE 2020)/6th International Conference on Information Systems Security and Privacy (ICISSP 2020), Valetta, Malta, 25–27 February, 2020

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.